

Mediapipe Human Pose Follow (PTZ)

1. Experiment Objective

PTZ pan-tilt + car body collaborative human pose following — MediaPipe Pose detects 33 landmarks, uses the mean center of all landmarks as the tracking target, keeps the tilt axis fixed (`fixed_pitch_deg` = -10°), and coordinates horizontal PD + body movement.

2. Experiment Principle

1. Key Terms

Term	Description
Visual Following	Pan-tilt tracking + coordinated body movement
MediaPipe Pose	33-landmark whole-body pose estimation; the landmark mean center is used as the tracking target
Fixed tilt angle (<code>fixed_pitch_deg</code>)	The tilt axis does not participate in PD tracking and is fixed at a specified angle (default -10°), because a human's vertical movement range is limited
Error sharing / angular-rate feedforward / distance follow	The core mechanisms of the three-layer coordinated control

2. Technical Architecture

Node chain:

State estimation layer → `pose_follow_ptz_node` → `/cmd_ptz_angle` (pan-tilt angle) + `/cmd_vel` (body velocity)

Three-layer coordinated control:

- **PTZ PD control (20 Hz):** the human center x pixel error → PD (`pd_kp` = 3.0, `pd_kd` = 1.5, the highest gains — human movement is large) → feedforward compensation → `/cmd_ptz_angle.x`. The tilt direction is fixed and not tracked; `fixed_pitch_deg` (-10°) is always output, avoiding tilt jitter caused by changes in human height.
- **Body angular-velocity PD control (10 Hz):** pan-tilt horizontal angle × `body_share` (0.5) → error shared with the body → PD (`angular_kp` = 0.8, `angular_kd` = 0.05) → limiting ±0.12 rad/s.
- **Distance-follow PD control (10 Hz):** deviation between the human bbox height and `target_pose_size_px` (320 px) → PD (`distance_kp` = 0.008, `distance_kd` = 0.003) → limiting ±0.20 m/s.

Lost-frame tolerance: no person for ≥5 consecutive frames → freeze the body.

3. Core Topics Quick Reference

Firmware Uplink (ESP32 → ROS2):

Topic Name	Type	QoS	Description
<code>/camera/image_raw</code>	sensor_msgs/Image	KEEP_LAST 1, BEST_EFFORT, VOLATILE	Raw image input from the ESP32 camera

Firmware Downlink (ROS2 → ESP32 via micro_ros_agent):

Topic Name	Type	QoS	Description
<code>/cmd_ptz_angle</code>	geometry_msgs/Vector3	Default (KEEP_LAST 10)	PTZ pan-tilt angle command (x: horizontal, y: tilt)
<code>/cmd_vel</code>	geometry_msgs/Twist	Default (KEEP_LAST 10)	Body velocity command (linear.x: forward/back, angular.z: rotation)

ROS2 Internal (published by the state estimation layer):

Topic Name	Type	QoS	Description
<code>/diagnostics</code>	diagnostic_msgs/DiagnosticArray	Default	Summary of each node's running status

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	☒
No-PTZ version	☐
Required peripherals	ESP32 camera (pan-tilt) + PTZ servo

2. Start Agent + Camera + Joint Model (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```

```

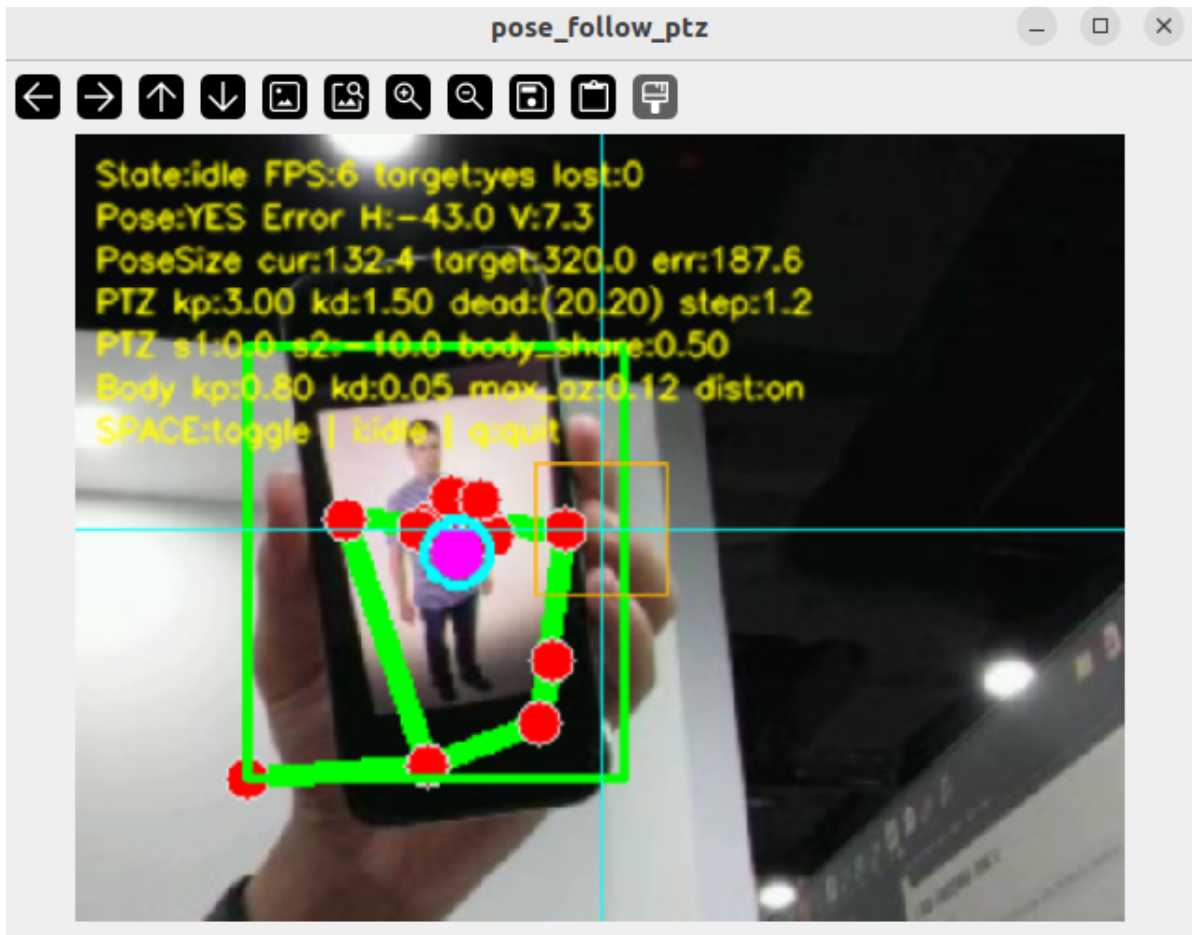
yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-15-04-46-269642-yahboom-579370
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [579411]
[INFO] [camera_driver-2]: process started with pid [579413]
[INFO] [robot_state_publisher-3]: process started with pid [579415]
[INFO] [joint_state_publisher-4]: process started with pid [579417]
[INFO] [ekf_node-5]: process started with pid [579419]
[micro_ros_agent-1] [1785222287.404236] info | UDPv4AgentLinux.cpp | init | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785222288.847816851] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785222288.848013042] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785222288.848028942] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848037122] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785222288.848043874] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785222288.848050530] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785222288.848057264] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785222288.848063747] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785222288.848070240] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848076888] [robot_state_publisher]: got segment rfwheel_Link
[joint_state_publisher-4] [INFO] [1785222289.372349515] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description
[camera_driver-2] [INFO] [1785222289.716161193] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[camera_driver-2] [WARN] [1785222292.785741414] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785222294.072701944] [esp32_ip_camera_publisher]: IP camera stream opened successfully

```

3. Start Human Pose Follow (Terminal 2)

Open a new terminal and run:

```
ros2 launch microros_v2_ptz_visual_control_pkg pose_follow_ptz.launch.py
```



After the node starts, the initial PTZ angle is published automatically, and the Space key toggles the idle/tracking state. At this point the pan-tilt is powered on but not yet in tracking mode.

4. Operation Instructions

Interaction:

Key	Function
Space	idle ↔ tracking toggle
i / I	Force idle
q / Q / ESC	Exit the program

Workflow:

1. After the program starts, it is in the **idle** state by default; MediaPipe pose detection is already running but **outputs no control commands**
2. Press **Space** to enter the **tracking** state; the pan-tilt tracks the human center horizontally, and the body turns + follows distance in coordination
3. Press **i** to return to idle, press **q** to exit

Safety and Edge Cases:

- No person ≥ 5 frames → freeze the body (`lost_frame_threshold=5`)
- Tilt fixed at `fixed_pitch_deg` (-10°), not involved in PD tracking
- Pan-tilt limiting: horizontal $[-90^\circ, 90^\circ]$, tilt $[-90^\circ, 20^\circ]$
- Velocity limiting: angular velocity ± 0.12 rad/s, linear velocity ± 0.20 m/s

5. How to Stop

Press `Ctrl+C` to stop Terminal 2 → Terminal 1 in order.

4. Experiment Observations

- **idle:** Pose detection runs but outputs no control; the frame shows the human landmarks and the center marker
- **tracking + human present:** The pan-tilt tracks the human center horizontally (purple dot), and the body turns quickly (`angular_kp=0.8`) + follows distance
- **Tilt not tracked:** always points at `fixed_pitch_deg` (-10°); changes in the person's height do not affect the pan-tilt
- **Human moves left/right:** high-gain PD (3.0/1.5) responds to the pan-tilt quickly, and the high body angular-velocity gain (0.8) turns in coordination
- **Human moves forward/back:** distance following based on the bbox size, up to 0.20 m/s
- **No person ≥ 5 frames:** the body freezes and the frame shows the lost count

5. Program Analysis

Source code paths:

- Launch file:
`~/microros_ws/src/microros_v2_ptz_visual_control_pkg/launch/pose_follow_ptz.launch.py`
- Node file:
`~/microros_ws/src/microros_v2_ptz_visual_control_pkg/microros_v2_ptz_visual_control_pkg/pose_follow_ptz_node.py`

1. Core Node Analysis

`PoseFollowPtzNode` inherits from `rcipy.node.Node` and internally contains three timer loops: image processing (15 Hz), PTZ control (20 Hz), body control (10 Hz), plus key polling (20 Hz).

Node initialization — `__init__`:

```
# Source file:
~/microros_ws/src/microros_v2_ptz_visual_control_pkg/microros_v2_ptz_visual_control_pkg/pose_follow_ptz_node.py

class PoseFollowPtzNode(Node):

    def __init__(self) -> None:
        super().__init__(DEFAULT_NODE_NAME)
        self._image_lock = threading.Lock()
        self._pose_lock = threading.Lock()

        self._latest_frame: Optional[np.ndarray] = None
        self._latest_seq = 0
        self._processed_seq = 0

        self._state = STATE_IDLE
        self._shutdown = False

        self._has_pose = False
        self._has_target = False
        self._lost_frames = 0
        self._target_x: Optional[float] = None
        self._target_y: Optional[float] = None
        self._target_size_px: Optional[float] = None

        self._h_deg = 0.0
        self._p_deg = 0.0

        self._prev_err_x = 0.0
        self._prev_err_y = 0.0
        self._prev_ctrl_t = 0.0
        self._last_sx = 0.0
        self._last_sy = 0.0

        self._prev_s1 = 0.0
        self._prev_s1_eff = 0.0
        self._prev_body_t = 0.0
        self._last_body_az = 0.0

        self._prev_size_err = 0.0
        self._prev_linear_t = 0.0

        self._measured_fps = 0.0
        self._fps_count = 0
        self._fps_last_t = time.perf_counter()
        self._win_ready = False
        self._last_print_t = 0.0

        self._declare_params()
        self._load_params()
```

```

        self._h_deg = float(self._init_h_deg)
        self._p_deg = clamp(float(self._fixed_pitch_deg), self._p_min,
self._p_max)

        self._pose_det = PoseDetector(
            detectionCon=float(self._pose_conf),
            trackCon=float(self._pose_track_conf),
        )

        video_qos = QoSProfile(
            history=QoSHistoryPolicy.KEEP_LAST,
            depth=1,
            reliability=QoSReliabilityPolicy.BEST_EFFORT,
            durability=QSDurabilityPolicy.VOLATILE,
        )
        self._img_sub = self.create_subscription(
            Image, self._img_topic, self._on_image, video_qos
        )
        self._ptz_pub = self.create_publisher(Vector3, self._ptz_topic, 10)
        self._cmd_pub = self.create_publisher(Twist, self._cmd_topic, 10)

        self._img_timer = self.create_timer(
            1.0 / max(float(self._img_rate), MINIMUM_TIMER_RATE_HZ),
self._process_image
        )
        self._ptz_timer = self.create_timer(
            1.0 / max(float(self._ptz_rate), MINIMUM_TIMER_RATE_HZ),
self._ptz_tick
        )
        self._body_timer = None
        self._rebuild_body_timer(False)

        self._key_timer = None
        if self._show_win:
            self._ensure_win()
            self._key_timer = self.create_timer(0.05, self._poll_keys)

        self._init_ptz_timer = None
        self._init_ptz_remain = 0
        if self._pub_init_ptz:
            self._start_init_ptz()

        self._param_cb =
self.add_on_set_parameters_callback(self._on_param_update)

        self.get_logger().info(
            f"Pose follow PTZ+body started. SPACE=toggle, i=idle, q=quit. "
            f"ptz={self._ptz_rate:.0f}Hz img={self._img_rate:.0f}Hz body=
{self._body_rate:.0f}Hz"
        )
    )

```

PTZ control core — `_ptz_tick` (fixed-pitch implementation):

```

# Source file:
~/microros_ws/src/microros_v2_ptz_visual_control_pkg/microros_v2_ptz_visual_cont
rol_pkg/pose_follow_ptz_node.py

```

```

def _ptz_tick(self) -> None:
    if self._state != STATE_TRACKING:
        return

    with self._pose_lock:
        tx = self._target_x
        ty = self._target_y
        has_target = self._has_target

    if not has_target or tx is None or ty is None:
        self._reset_ptz_pd()
        return

    icx = float(self._proc_w) * 0.5
    icy = float(self._proc_h) * 0.5
    pex = float(tx - icx)
    pey = float(ty - icy)

    cx = abs(pex) < self._db_x
    cy = abs(pey) < self._db_y
    if cx:
        pex = 0.0
    if cy:
        pey = 0.0

    n = time.perf_counter()
    dt = n - self._prev_ctrl_t if self._prev_ctrl_t > 0.0 else 1.0 /
max(float(self._ptz_rate), MINIMUM_TIMER_RATE_HZ)
    dt = max(dt, 1e-3)

    nex = pex / icx
    ney = pey / icy
    dx = (nex - self._prev_err_x) / dt
    dy = (ney - self._prev_err_y) / dt

    ff = self._body_ff_gain * self._last_body_az
    rx = clamp((self._kp * nex + self._kd * dx - ff) * self._max_step, -
self._max_step, self._max_step)
    ry = 0.0 # pitch is fixed: no PD control output

    a = self._step_alpha
    sx = (1.0 - a) * self._last_sx + a * rx
    sy = (1.0 - a) * self._last_sy + a * ry
    if cx:
        sx = 0.0
        nex = 0.0
    if cy:
        sy = 0.0
        ney = 0.0

    self._prev_err_x = nex
    self._prev_err_y = ney
    self._prev_ctrl_t = n
    self._last_sx = sx
    self._last_sy = sy

    if self._rev_h:

```

```

        sx = -sx
        dh = clamp(self._h_deg + sx, self._h_min, self._h_max)
        dp = clamp(self._fixed_pitch_deg, self._p_min, self._p_max)

        sa = self._ptz_alpha
        self._h_deg = clamp((1.0 - sa) * self._h_deg + sa * dh, self._h_min,
self._h_max)
        self._p_deg = dp

        self._pub_ptz()
        self._log_ctrl(pex, pey)

```

The fixed-pitch key lines: `ry = 0.0` and `dp = clamp(self._fixed_pitch_deg, self._p_min, self._p_max)` — the tilt direction performs no PD error computation and always outputs the `fixed_pitch_deg` angle.

2. Key Parameters

Parameter definition files:

- Launch file:
`~/microros_ws/src/microros_v2_ptz_visual_control_pkg/launch/pose_follow_ptz.launch.py`
- Node file:
`~/microros_ws/src/microros_v2_ptz_visual_control_pkg/microros_v2_ptz_visual_control_pkg/pose_follow_ptz_node.py`

Pan-tilt PTZ control parameters:

Parameter	Type	Default Value	Description
<code>pd_kp</code>	float	3.0	Pan-tilt horizontal PD proportional gain (highest, human movement is large)
<code>pd_kd</code>	float	1.5	Pan-tilt horizontal PD derivative gain
<code>ptz_control_rate</code>	float	20.0 Hz	Pan-tilt control rate
<code>fixed_pitch_deg</code>	float	-10.0°	Fixed tilt angle (not involved in PD tracking)
<code>deadband_x</code>	float	20.0 px	Horizontal deadband (no response near the center)
<code>max_ptz_step</code>	float	1.2°	Maximum single step angle
<code>ptz_step_smooth_alpha</code>	float	0.35	Step smoothing coefficient
<code>ptz_smooth_alpha</code>	float	0.85	Angle output smoothing coefficient

Body control parameters:

Parameter	Type	Default Value	Description
<code>body_control_rate</code>	float	10.0 Hz	Body control rate
<code>angular_kp</code>	float	0.8	Body angular-velocity P gain (high, for fast turning)
<code>angular_kd</code>	float	0.05	Body angular-velocity D gain
<code>body_share</code>	float	0.5	Ratio of the pan-tilt error allocated to the body
<code>body_ff_gain</code>	float	1.3	Angular-rate feedforward gain
<code>max_angular_z</code>	float	0.12 rad/s	Maximum angular velocity

Distance-follow parameters:

Parameter	Type	Default Value	Description
<code>enable_distance_follow</code>	bool	true	Whether to enable distance following
<code>target_pose_size_px</code>	float	320.0 px	Distance-follow target human bbox height
<code>distance_kp</code>	float	0.008	Distance PD proportional gain
<code>distance_kd</code>	float	0.003	Distance PD derivative gain
<code>max_linear_x</code>	float	0.20 m/s	Maximum linear velocity
<code>distance_tolerance_pose_size_px</code>	float	10.0 px	Distance deadband

Pose detection parameters:

Parameter	Type	Default Value	Description
<code>pose_detection_confidence</code>	float	0.5	MediaPipe pose detection confidence threshold
<code>pose_tracking_confidence</code>	float	0.5	MediaPipe pose tracking confidence threshold
<code>lost_frame_threshold</code>	int	5	Consecutive lost-frame tolerance count

6. Frequently Asked Questions

Problem	Cause	Solution
No vertical tracking	The tilt axis is fixed by design	Adjust the fixed tilt angle via <code>fixed_pitch_deg</code>
Obvious lag	Pose 33 landmarks + three-layer control delay	Reduce the <code>processing_width / processing_height</code> resolution
The body turns too fast	<code>angular_kp</code> = 0.8 is a high gain	Reduce <code>angular_kp</code> (e.g., 0.4)
Distance following is not responsive	<code>distance_kp</code> = 0.008 is a low gain	Increase <code>distance_kp</code> appropriately